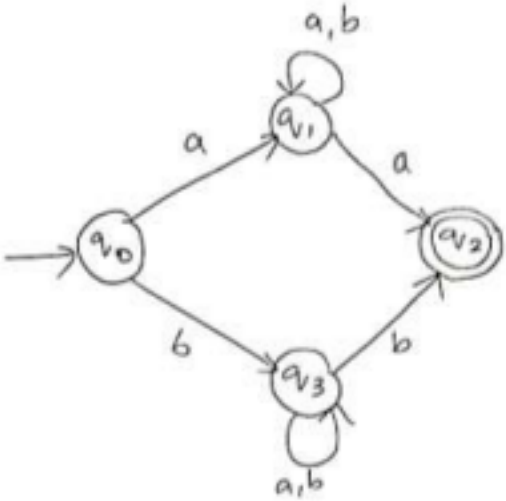
Deva Dharshini
192472350

SAVEETHA SCHOOL OF ENGINEERING

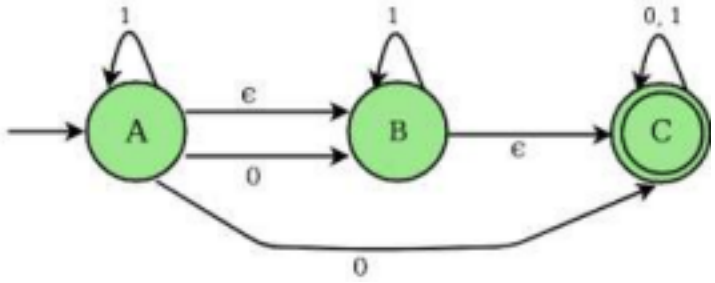
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

COMPUTER SCIENCE AND ENGINEERING

CSA13 – THEORY OF COMPUTATION

	ASSIGNMENT – 1	CO	BL
1	Design a Deterministic Finite Automata (DFA) to accept strings that start and end with same symbols over □□={0,1}. Write the formal definition of the DFA. Also show that the string **010100** is accepted by the DFA	1	3
2	Design NFA to accept string having 101 as substring	1	3
3	Apply the principle of mathematical induction and prove that for all positive integers n, the statement given below is true. $\sum_{□□=1}^{□□} □□(□□+1) = \frac{□□(□□+1)(□□+2)}{3}$	1	1
4	Consider the NFA given below which accepts the language representing the set of all strings that start and end with the same symbol over the alphabet □□={a,b}. Construct a DFA that represents the same language 	1	3

Deva Dharshini
192472350

5	A NFA with ε-transitions is given below. Find ε-closure for each state and construct an equivalent NFA without ε-transitions that accepts the same language. Draw its transition table and mark the initial and final states 	1	3

Deva Dharshini
192472350

QUESTION 1: DFA for strings that start and end with the same symbol over Σ = {0, 1}

1. Introduction and basic terminology

Before designing the machine, these terms are needed:

- **Alphabet (Σ):** A finite, non-empty set of symbols. Here Σ = {0, 1}.
- **String:** A finite sequence of symbols from Σ, such as 0, 01 or 1101.
- **Empty string (ε):** The string of length zero.
- **Σ*:** The set of all strings over Σ, including ε.
- **Language:** Any subset of Σ*.
- **Finite automaton:** A mathematical model of computation with a finite number of states and no extra memory. It reads the input one symbol at a time, from left to right.

A **Deterministic Finite Automaton (DFA)** is a finite automaton in which, for every state and every input symbol, there is **exactly one** transition. There is no choice and no ambiguity: the next state is fully determined by the current state and the current input symbol.

2. Formal definition of a DFA

A DFA is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where:

1. Q is a finite set of states.
2. Σ is a finite input alphabet.
3. $\delta : Q \times \Sigma \rightarrow Q$ is the transition function.
4. $q_0 \in Q$ is the start (initial) state.
5. $F \subseteq Q$ is the set of final (accepting) states.

Extended transition function $\delta^* : Q \times \Sigma^* \rightarrow Q$ is defined recursively:

- $\delta^*(q, \varepsilon) = q$
- $\delta^*(q, wa) = \delta(\delta^*(q, w), a)$, for any string w and symbol a

Language accepted by M: $L(M) = \{ w \in \Sigma^* \mid \delta^*(q_0, w) \in F \}$

3. Analysis of the given problem

The language to be accepted is:

$L = \{ w \in \{0,1\}^* \mid$ w is non-empty and the first symbol of w equals the last symbol of w $\}$

Strings in L: 0, 1, 00, 11, 010, 101, 0110, 1001, 01010, 010100, 1011

Strings not in L: ε, 01, 10, 001, 110, 0101, 1010

Points to note:

1. A string of length 1 (such as "0" or "1") has its first and last symbol as the same character, so it **is** accepted.
2. The empty string ε has no first symbol and no last symbol, so it is **not** accepted.
3. For longer strings, the symbols in the middle do not matter. Only the first and last symbols matter.

4. Design idea

To decide whether a string belongs to L, the machine needs to remember only two pieces of information:

1. What was the first symbol? (0 or 1)
2. What is the most recently read symbol? (0 or 1)

This gives 2 × 2 = 4 combinations, plus one start state for "nothing read yet". So the DFA has 5 states.

5. Meaning of each state

Deva Dharshini
192472350

State	Meaning (invariant)	Final?
q0	Nothing has been read yet (start state)	No
q1	First symbol was 0 and the last symbol read is 0	Yes
q2	First symbol was 0 and the last symbol read is 1	No
q3	First symbol was 1 and the last symbol read is 1	Yes
q4	First symbol was 1 and the last symbol read is 0	No

The final states are exactly those where "first symbol = last symbol", i.e. q1 and q3.

6. Formal definition of the designed DFA

$M = (Q, \Sigma, \delta, q_0, F)$, where:

- $Q = \{q0, q1, q2, q3, q4\}$
- $\Sigma = \{0, 1\}$
- Start state = q0
- $F = \{q1, q3\}$
- δ is defined below

7. Transition function δ

- $\delta(q0, 0) = q1$
- $\delta(q0, 1) = q3$
- $\delta(q1, 0) = q1$
- $\delta(q1, 1) = q2$
- $\delta(q2, 0) = q1$
- $\delta(q2, 1) = q2$
- $\delta(q3, 0) = q4$
- $\delta(q3, 1) = q3$
- $\delta(q4, 0) = q4$
- $\delta(q4, 1) = q3$

Transition table (→ marks the start state, * marks final states):

State	Input 0	Input 1
→ q0	q1	q3
* q1	q1	q2
q2	q1	q2
* q3	q4	q3
q4	q4	q3

8. Explanation of every transition

From q0 (start):

- On 0, the first symbol is 0 and the last symbol is also 0, so go to q1.
- On 1, the first symbol is 1 and the last symbol is also 1, so go to q3.

From q1 (first = 0, last = 0):

- On 0, the last symbol is still 0, so stay in q1.
- On 1, the last symbol becomes 1 while the first is still 0, so go to q2.

From q2 (first = 0, last = 1):

- On 0, the last symbol becomes 0 again, matching the first, so go back to q1.
- On 1, the last symbol is still 1, so stay in q2.

From q3 (first = 1, last = 1):

- On 1, the last symbol is still 1, so stay in q3.
- On 0, the last symbol becomes 0 while the first is 1, so go to q4.

From q4 (first = 1, last = 0):

- On 0, the last symbol is still 0, so stay in q4.
- On 1, the last symbol becomes 1, matching the first, so go back to q3.

9. Transition diagram (description)

- The start arrow points to q0.
- q0 goes to q1 on 0 and to q3 on 1.
- q1 has a self-loop on 0, goes to q2 on 1, and q2 returns to q1 on 0.
- q2 has a self-loop on 1.
- q3 has a self-loop on 1, goes to q4 on 0, and q4 returns to q3 on 1.
- q4 has a self-loop on 0.
- q1 and q3 are drawn with **double circles** because they are final states.

10. Proof of correctness (why the DFA works)

Claim: After reading a non-empty string w, the DFA is in:

- q1 if w starts with 0 and ends with 0
- q2 if w starts with 0 and ends with 1
- q3 if w starts with 1 and ends with 1
- q4 if w starts with 1 and ends with 0

Proof by induction on the length of w:

- **Base (|w| = 1):** For w = 0, $\delta(q0,0) = q1$ (starts 0, ends 0). For w = 1, $\delta(q0,1) = q3$ (starts 1, ends 1). ✓
- **Step:** Assume the claim for a string w of length n, and let a be the next symbol.
  - If w starts with 0, the DFA is in q1 or q2. On input 0 both go to q1, and on input 1 both go to q2. This is correct because the first symbol stays 0 and the last symbol becomes a.
  - If w starts with 1, the DFA is in q3 or q4. On input 1 both go to q3, and on input 0 both go to q4. ✓

So the DFA is in a final state (q1 or q3) exactly when the first and last symbols are equal, and the DFA is correct.

11. Acceptance of the string 010100

The string is 0 1 0 1 0 0 (length 6).

Step	Current state	Symbol read	δ(current, symbol) = Next state
1	q0	0	$\delta(q0, 0) = q1$
2	q1	1	$\delta(q1, 1) = q2$
3	q2	0	$\delta(q2, 0) = q1$
4	q1	1	$\delta(q1, 1) = q2$
5	q2	0	$\delta(q2, 0) = q1$
6	q1	0	$\delta(q1, 0) = q1$

Path: q0 → q1 → q2 → q1 → q2 → q1 → q1

Using the extended transition function:

- $\delta^*(q0, 0) = q1$

- $\delta^*(q0, 01) = \delta(q1, 1) = q2$
- $\delta^*(q0, 010) = \delta(q2, 0) = q1$
- $\delta^*(q0, 0101) = \delta(q1, 1) = q2$
- $\delta^*(q0, 01010) = \delta(q2, 0) = q1$
- $\delta^*(q0, 010100) = \delta(q1, 0) =$ **q1**

Since $q1 \in F$, the string 010100 is accepted. This agrees with the definition, because it starts with 0 and ends with 0.

12. More examples

String	Path	Result
0	q0 → q1	Accepted
1	q0 → q3	Accepted
01	q0 → q1 → q2	Rejected
10	q0 → q3 → q4	Rejected
101	q0 → q3 → q4 → q3	Accepted
0110	q0 → q1 → q2 → q2 → q1	Accepted
1001	q0 → q3 → q4 → q4 → q3	Accepted
0101	q0 → q1 → q2 → q1 → q2	Rejected
ε	q0 (not final)	Rejected

13. Regular expression for the language

L = 0 + 1 + 0(0+1)*0 + 1(0+1)*1

Here 0 and 1 account for the one-symbol strings. The other two parts account for the longer strings.

14. Minimality of the DFA

The five states are pairwise distinguishable:

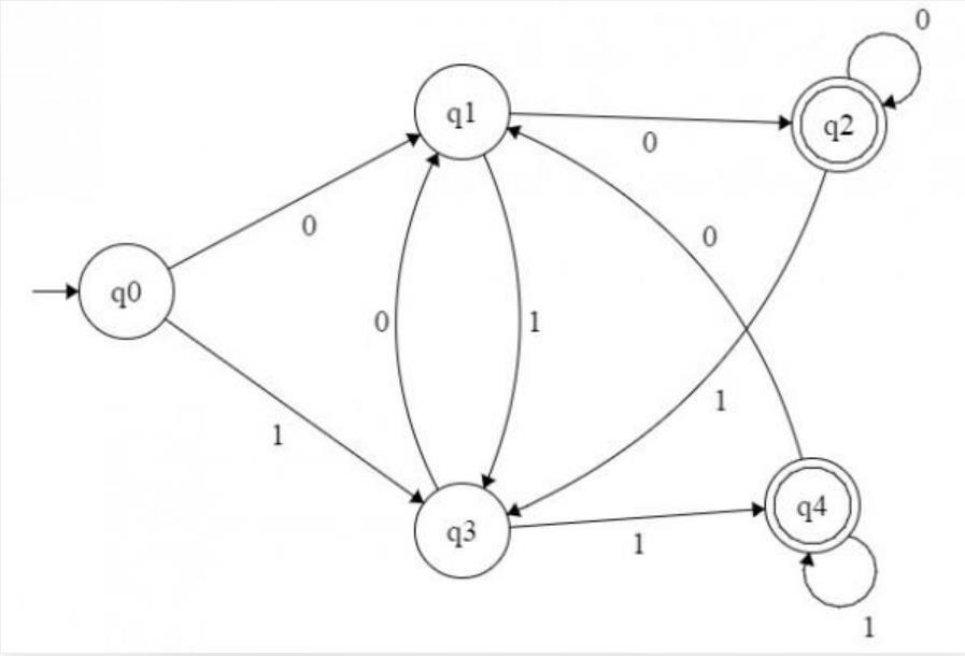
- q1 and q3 are final, while q0, q2 and q4 are non-final.
- q1 vs q3: on input 1, q1 goes to q2 (non-final) but q3 goes to q3 (final).
- q0 vs q2: on input 1, q0 goes to q3 (final) but q2 goes to q2 (non-final).
- q0 vs q4: on input 0, q0 goes to q1 (final) but q4 goes to q4 (non-final).
- q2 vs q4: on input 0, q2 goes to q1 (final) but q4 goes to q4 (non-final).

Since no two states are equivalent, this 5-state DFA is minimal.

15. Conclusion

The 5-state DFA M = ({q0,q1,q2,q3,q4}, {0,1}, δ, q0, {q1,q3}) accepts exactly the strings over {0,1} that start and end with the same symbol. It accepts the string 010100 by ending in the final state q1.

Deva Dharshini
192472350



QUESTION 2: NFA to accept strings having 101 as a substring

1. Introduction

A Nondeterministic Finite Automaton (NFA) is a finite automaton where, from a given state on a given input symbol, the machine may have zero, one, or several possible next states. This allows the machine to "guess" the correct path.

Acceptance rule: A string is accepted if at least one computation path, from the start state through the entire string, ends in a final state. It is rejected only if every path fails (ends in a non-final state or gets stuck).

2. Formal definition of an NFA

An NFA is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where:

1. Q is a finite set of states.
2. Σ is the input alphabet.
3. **$\delta : Q \times \Sigma \rightarrow 2^Q$** is the transition function, which maps to a **set** of states (possibly empty).
4. **q_0** is the start state.
5. **$F \subseteq Q$** is the set of final states.

Extended transition function:

- $\delta^*(q, \varepsilon) = \{q\}$
- $\delta^*(q, wa) = \bigcup \{ \delta(p, a) : p \in \delta^*(q, w) \}$

Language accepted: $L(N) = \{ w \mid \delta^*(q_0, w) \cap F \neq \emptyset \}$

3. Differences between DFA and NFA

Feature	**DFA**	**NFA**
Next state	Exactly one	Zero, one or many
Transition function	$\delta : Q \times \Sigma \rightarrow Q$	$\delta : Q \times \Sigma \rightarrow 2^Q$
Acceptance	The single path ends in a final state	At least one path ends in a final state
Design	Often large and complex	Often small and simple
Power	Recognises regular languages	Recognises the same regular languages

4. Analysis of the problem

Deva Dharshini
192472350

The language is:

L = { w ∈ {0,1}* | w = x101y for some strings x, y ∈ {0,1}* }

Strings in L: 101, 0101, 1010, 1101, 10101, 11011, 0010100

Strings not in L: ε, 0, 1, 10, 11, 100, 110, 1001, 0110, 1100

5. Design strategy

The NFA has three logical parts:

1. **Skipping the prefix:** A start state q0 that loops on both 0 and 1. This lets the NFA ignore anything before 101.
2. **Matching "101":** A chain of three transitions: 1 (q0→q1), then 0 (q1→q2), then 1 (q2→q3).
3. **Skipping the suffix:** A final state q3 that loops on both 0 and 1. Once 101 has been seen, the rest of the string does not matter.

Role of nondeterminism: When the NFA is in q0 and reads a 1, it does not know whether this 1 is the start of 101. So it takes **both** choices: stay in q0, or move to q1.

6. Meaning of each state

State	Meaning
q0	Start state. Still searching, and nothing useful matched so far
q1	The first "1" of the pattern 101 has been matched
q2	The prefix "10" of the pattern has been matched
q3	The full pattern "101" has been matched (final state)

7. Formal definition

N = (Q, Σ, δ, q_0, F)

- Q = {q0, q1, q2, q3}
- Σ = {0, 1}
- Start state: q0
- F = {q3}

8. Transition function

- δ(q0, 0) = {q0}
- δ(q0, 1) = {q0, q1}
- δ(q1, 0) = {q2}
- δ(q1, 1) = ∅
- δ(q2, 0) = ∅
- δ(q2, 1) = {q3}
- δ(q3, 0) = {q3}
- δ(q3, 1) = {q3}

Transition table:

State	0	1
→ q0	{q0}	{q0, q1}
q1	{q2}	∅
q2	∅	{q3}
* q3	{q3}	{q3}

9. Transition diagram (description)

q0 has a self-loop labelled 0,1.

- There is an arrow labelled 1 from q0 to q1.
- There is an arrow labelled 0 from q1 to q2.
- There is an arrow labelled 1 from q2 to q3.
- q3 is a double circle with a self-loop labelled 0,1.

10. Working with examples

Example 1: 101 (accepted)

- Start: {q0}
- Read 1: {q0, q1}
- Read 0: from q0 → q0, from q1 → q2, giving {q0, q2}
- Read 1: from q0 → {q0, q1}, from q2 → q3, giving {q0, q1, q3}
- The final set contains q3, so the string is **accepted**.

Example 2: 0101 (accepted)

- Start: {q0}
- Read 0: {q0}
- Read 1: {q0, q1}
- Read 0: {q0, q2}
- Read 1: {q0, q1, q3}
- q3 is present, so the string is **accepted**.

Example 3: 1100 (rejected)

- Start: {q0}
- Read 1: {q0, q1}
- Read 1: q0 → {q0, q1}, q1 has no move on 1, giving {q0, q1}
- Read 0: q0 → q0, q1 → q2, giving {q0, q2}
- Read 0: q0 → q0, q2 has no move on 0, giving {q0}
- The final set {q0} contains no final state, so the string is **rejected**.

Example 4: 0110 (rejected)

- {q0} → (0) {q0} → (1) {q0,q1} → (1) {q0,q1} → (0) {q0,q2}
- q3 is not present, so the string is **rejected**.

Example 5: 11011 (accepted)

- {q0} → (1) {q0,q1} → (1) {q0,q1} → (0) {q0,q2} → (1) {q0,q1,q3} → (1) {q0,q1,q3}
- q3 is present, so the string is **accepted**.

11. Proof of correctness

Claim: w ∈ L(N) if and only if w contains 101 as a substring.

- **(⇐)** Suppose w = x101y. The NFA stays in q0 while reading x. It then moves q0 → q1 → q2 → q3 while reading 1, 0, 1. It then stays in q3 while reading y. This path ends in q3, which is final, so w is accepted.
- **(⇒)** Suppose w is accepted. Then some path reaches q3. The only way into q3 is the edge q2 –1→ q3, and the only way into q2 is q1 –0→ q2, and the only way into q1 is q0 –1→ q1. So the path must have read 1, 0, 1 consecutively, which means 101 is a substring of w.

12. Equivalent DFA (optional extension, by subset construction)

DFA state	**0**	**1**
→ {q0}	{q0}	{q0,q1}

Deva Dharshini
192472350

DFA state	0	1
{q0,q1}	{q0,q2}	{q0,q1}
{q0,q2}	{q0}	{q0,q1,q3}
* {q0,q1,q3}	{q0,q2,q3}	{q0,q1,q3}
* {q0,q2,q3}	{q0,q3}	{q0,q1,q3}
* {q0,q3}	{q0,q3}	{q0,q1,q3}

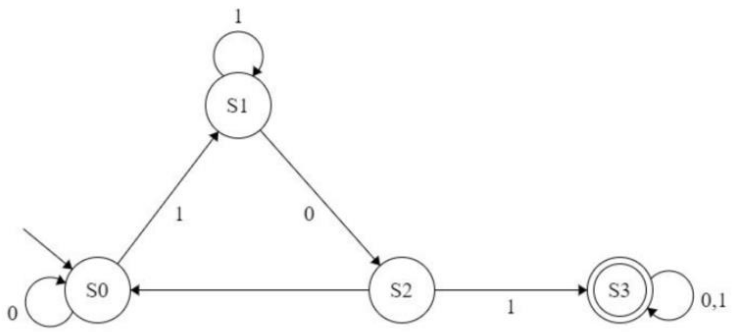
The three final DFA states all behave the same (every continuation is accepted), so they can be merged into one trap-like accepting state. That gives a minimal DFA with 4 states. This shows that the NFA is a compact way of describing the language.

13. Regular expression

L = (0+1)* 101 (0+1)*

14. Conclusion

The 4-state NFA N = ({q0,q1,q2,q3}, {0,1}, δ, q0, {q3}) accepts exactly the strings over {0,1} that contain 101 as a substring. Nondeterminism in q0 on input 1 makes the design simple.



Deva Dharshini
192472350

QUESTION 3: Proof by mathematical induction

1. Statement to be proved

For every positive integer n:

Σ (k = 1 to n) k(k+1) = n(n+1)(n+2) / 3

That is: $1\cdot2 + 2\cdot3 + 3\cdot4 + \dots + n(n+1) = n(n+1)(n+2)/3$.

2. Principle of mathematical induction

Let P(n) be a statement about the positive integer n. If

1. P(1) is true (basis step), and
2. for every $k \geq 1$, P(k) true implies P(k+1) true (inductive step),

then P(n) is true for all positive integers n.

Why it works (intuition): P(1) is true. By step 2, P(1) $\Rightarrow$ P(2), then P(2) $\Rightarrow$ P(3), and so on. It is like an infinite row of dominoes. If the first falls, and each falling domino knocks over the next one, then every domino falls.

3. Defining P(n)

P(n): $1\cdot2 + 2\cdot3 + \dots + n(n+1) = n(n+1)(n+2)/3$

4. Step 1: Basis step (n = 1)

- LHS = Σ (k=1 to 1) k(k+1) = $1\cdot(1+1) = 1\cdot2 = 2$
- RHS = $1\cdot(1+1)\cdot(1+2)/3 = (1\cdot2\cdot3)/3 = 6/3 = 2$

Since LHS = RHS, P(1) is true.

5. Step 2: Induction hypothesis

Assume P(k) is true for some arbitrary positive integer k:

$1\cdot2 + 2\cdot3 + \dots + k(k+1) = k(k+1)(k+2)/3$ …… (1)

6. Step 3: Inductive step (prove P(k+1))

To prove:

$1\cdot2 + 2\cdot3 + \dots + k(k+1) + (k+1)(k+2) = (k+1)(k+2)(k+3)/3$

Proof: Start from the left-hand side.

LHS = $[1\cdot2 + 2\cdot3 + \dots + k(k+1)] + (k+1)(k+2)$

Use equation (1) to replace the bracket:

LHS = $k(k+1)(k+2)/3 + (k+1)(k+2)$

Take the common factor (k+1)(k+2):

LHS = $(k+1)(k+2) \cdot [\, k/3 + 1 \,]$

Write 1 as 3/3 inside the bracket:

LHS = $(k+1)(k+2) \cdot [\, (k + 3)/3 \,]$

LHS = **(**k+1)(k+2)(k+3)/3

This equals the right-hand side of P(k+1). Hence P(k+1) is true whenever P(k) is true.

7. Conclusion

- The basis step shows P(1) is true.
- The inductive step shows P(k) $\Rightarrow$ P(k+1) for every $k \geq 1$.

By the **principle of mathematical induction**, P(n) is true for all positive integers n:

Σ (k=1 to n) k(k+1) = n(n+1)(n+2)/3.

8. Numerical verification

n	Sum $1\cdot2 + \dots + n(n+1)$	n(n+1)(n+2)/3	Equal?
1	2	$1\cdot2\cdot3/3 = 2$	✓

2	2 + 6 = 8	2·3·4/3 = 8	✓
3	8 + 12 = 20	3·4·5/3 = 20	✓
4	20 + 20 = 40	4·5·6/3 = 40	✓
5	40 + 30 = 70	5·6·7/3 = 70	✓

9. Alternative direct proof (for cross-checking)

Expand $k(k+1) = k^2 + k$. Then:

$\Sigma\, k(k+1) = \Sigma\, k^2 + \Sigma\, k = n(n+1)(2n+1)/6 + n(n+1)/2$

Take the common factor $n(n+1)/6$:

$= n(n+1)/6 \cdot [\,(2n+1) + 3\,]$

$= n(n+1)(2n+4)/6$

$= n(n+1)\cdot 2(n+2)/6$

$=$ **$n(n+1)(n+2)/3$** ✓

This confirms the result obtained by induction.

QUESTION 4: Construct a DFA from the given NFA (strings starting and ending with the same symbol over {a, b})

1. Introduction

A fundamental theorem of automata theory states: **for every NFA there exists an equivalent DFA that accepts the same language**. Nondeterminism does not increase the power of finite automata. It only makes the design easier. The standard method for converting an NFA to a DFA is the **subset construction** (also called the power-set construction).

2. The given NFA (read from the diagram)

- **States:** Q = {q0, q1, q2, q3}
- **Alphabet:** Σ = {a, b}
- **Start state:** q0
- **Final state:** F = {q2}
- **Transitions:**
  - q0 –a→ q1
  - q0 –b→ q3
  - q1 –a,b→ q1 (self-loop)
  - q1 –a→ q2
  - q3 –a,b→ q3 (self-loop)
  - q3 –b→ q2

How the NFA works:

- The upper branch (via q1) handles strings that **start with a**. The NFA loops in q1 over the middle part and then guesses the last symbol is "a" to move to q2.
- The lower branch (via q3) handles strings that **start with b**. The NFA loops in q3 and then guesses the last symbol is "b" to move to q2.

NFA transition table:

State	a	b
→ q0	{q1}	{q3}
q1	{q1, q2}	{q1}

State	a	b
* q2	∅	∅
q3	{q3}	{q3, q2}

3. Subset construction algorithm

Input: NFA N = (Q, Σ, δ, q_0, F).

Output: DFA D = (Q', Σ, δ', q_0', F').

1. **Start state:** q_0' = {q_0}.
2. **Transitions:** For a DFA state S ⊆ Q and symbol x ∈ Σ, δ'(S, x) = ⋃ over p ∈ S of δ(p, x).
3. **New states:** Whenever δ'(S, x) is a set not yet seen, add it to Q' and process it.
4. **Repeat** until no new states appear.
5. **Final states:** F' = { S ∈ Q' | S ∩ F ≠ ∅ }.

Note: In the worst case the DFA could have $2^4 = 16$ states for 4 NFA states. In practice only the **reachable** subsets are constructed, which is usually far fewer.

4. Step-by-step construction

Step 1: Start state {q0}

- δ'({q0}, a) = δ(q0, a) = {q1} → new state {q1}
- δ'({q0}, b) = δ(q0, b) = {q3} → new state {q3}

Step 2: State {q1}

- δ'({q1}, a) = δ(q1, a) = {q1, q2} → new state {q1, q2}
- δ'({q1}, b) = δ(q1, b) = {q1} → already present

Step 3: State {q3}

- δ'({q3}, a) = δ(q3, a) = {q3} → already present
- δ'({q3}, b) = δ(q3, b) = {q3, q2} → new state {q3, q2}

Step 4: State {q1, q2}

- δ'({q1,q2}, a) = δ(q1,a) ∪ δ(q2,a) = {q1,q2} ∪ ∅ = {q1, q2} → already present
- δ'({q1,q2}, b) = δ(q1,b) ∪ δ(q2,b) = {q1} ∪ ∅ = {q1} → already present

Step 5: State {q3, q2}

- δ'({q3,q2}, a) = δ(q3,a) ∪ δ(q2,a) = {q3} ∪ ∅ = {q3} → already present
- δ'({q3,q2}, b) = δ(q3,b) ∪ δ(q2,b) = {q3,q2} ∪ ∅ = {q3, q2} → already present

No new states appear, so the construction is complete. The reachable DFA states are:

{q0}, {q1}, {q3}, {q1,q2}, {q3,q2}

The other subsets (such as ∅, {q2}, {q0,q1} and so on) are unreachable and are **not** included.

5. Resulting DFA transition table

DFA state	a	b
→ {q0}	{q1}	{q3}
{q1}	{q1, q2}	{q1}
{q3}	{q3}	{q3, q2}
* {q1, q2}	{q1, q2}	{q1}
* {q3, q2}	{q3}	{q3, q2}

Final states: Every DFA state containing q2 is final. These are **{q1,q2}** and **{q3,q2}**.

6. Renaming the states

Let A = {q0}, B = {q1}, C = {q3}, D = {q1,q2}, E = {q3,q2}.

Deva Dharshini
192472350

State	a	b
→ A	B	C
B	D	B
C	C	E
* D	D	B
* E	C	E

7. Formal definition of the DFA

D = (Q', Σ, δ', A, F') where

- Q' = {A, B, C, D, E}
- Σ = {a, b}
- Start state = A
- F' = {D, E}
- δ' is as in the table above

8. Transition diagram (description)

- Start arrow into **A**.
- A –a→ B and A –b→ C.
- B –a→ D and B has a self-loop on b.
- C has a self-loop on a, and C –b→ E.
- D (double circle) has a self-loop on a, and D –b→ B.
- E (double circle) has a self-loop on b, and E –a→ C.

9. Meaning of the DFA states

State	Meaning
A	Nothing read yet
B	Started with a, and the last symbol read is b (or only the first a has been read)
C	Started with b, and the last symbol read is a (or only the first b has been read)
D	Started with a, and the last symbol read is a, with length ≥ 2 (accepting)
E	Started with b, and the last symbol read is b, with length ≥ 2 (accepting)

Note that this machine requires the string to have **length at least 2**, because the NFA needs one move to enter q1 or q3 and another to enter q2. So the single-symbol strings "a" and "b" are not accepted by this NFA or DFA.

10. Verification with sample strings

String	Path in DFA	Final?	Result
aa	A → B → D	D is final	Accepted ✓
aba	A → B → B → D	D is final	Accepted ✓
abba	A → B → B → B → D	D is final	Accepted ✓
bb	A → C → E	E is final	Accepted ✓
bab	A → C → C → E	E is final	Accepted ✓
baab	A → C → C → C → E	E is final	Accepted ✓
ab	A → B → B	B is not final	Rejected ✓

ba	A → C → C	C is not final	Rejected ✓
abab	A → B → D → B → B	B is not final	Rejected ✓

11. Proof that the DFA is equivalent to the NFA

By the subset construction theorem, $\delta'^*(\{q_0\}, w) = \delta^*(q_0, w)$ for every string w. So the DFA ends in the set of all NFA states reachable on w. This set contains q2 exactly when the NFA has an accepting path, so L(D) = L(N).

12. Regular expression

L = a(a+b)*a + b(a+b)*b

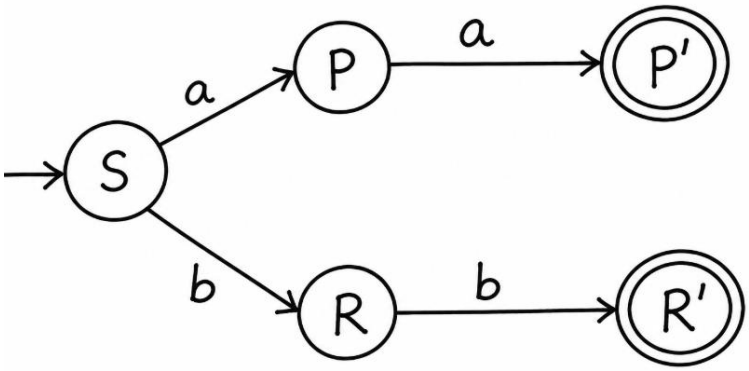
13. Minimality check

- D and E are final. B, C and A are non-final.
- D vs E: on input a, D goes to D (final) and E goes to C (non-final). So they are distinguishable.
- B vs C: on input a, B goes to D (final) and C goes to C (non-final). So they are distinguishable.
- A vs B: on input a, A goes to B (non-final) and B goes to D (final). So they are distinguishable.
- A vs C: on input b, A goes to C (non-final) and C goes to E (final). So they are distinguishable.

All five states are distinguishable, so this DFA is minimal.

14. Conclusion

Using the subset construction, the 4-state NFA is converted into an equivalent 5-state DFA with start state {q0} and final states {q1,q2} and {q3,q2}. It accepts exactly the strings over {a,b}, of length at least 2, that start and end with the same symbol.



QUESTION 5: ε-closure and conversion of an ε-NFA to an NFA without ε-transitions

1. Introduction

An NFA with ε-transitions (ε-NFA) is an NFA that is allowed to change state without consuming any input symbol. These "free moves" are called ε-transitions. They make it easy to combine automata (for example in the Thompson construction for regular expressions), but they are not essential. Every ε-NFA can be converted into an equivalent NFA without ε-**m**oves, and the two accept exactly the same language.

2. Formal definition of an ε-NFA

Deva Dharshini
192472350

An ε-NFA is a 5-tuple N = (Q, Σ, δ, q_0, F) where

- Q is a finite set of states,
- Σ is the input alphabet,
- δ : Q × (Σ ∪ {ε}) → 2^Q is the transition function,
- q_0 is the start state,
- F ⊆ Q is the set of final states.

3. Definition of ε-closure

The **ε-closure of a state q**, written **ε-closure(q)**, is the set of all states that can be reached from q by following **zero or more** ε-transitions.

Rules:

1. q ∈ ε-closure(q), because zero ε-moves are allowed.
2. If p ∈ ε-closure(q) and there is an ε-transition p –ε→ r, then r ∈ ε-closure(q).

For a set of states S, ε-closure(S) = ⋃ over q ∈ S of ε-closure(q).

4. The given ε-NFA (read from the diagram)

- **States:** Q = {A, B, C}
- **Alphabet:** Σ = {0, 1}
- **Start state:** A
- **Final state:** C
- **Transitions:**
  - A –1→ A (self-loop)
  - A –ε→ B
  - A –0→ B
  - A –0→ C
  - B –1→ B (self-loop)
  - B –ε→ C
  - C –0,1→ C (self-loop)

Transition table of the ε-NFA:

State	0	1	ε
→ A	{B, C}	{A}	{B}
B	∅	{B}	{C}
* C	{C}	{C}	∅

5. Step 1: Finding the ε-closure of each state

ε-closure(A):

- A is in its own closure.
- A –ε→ B, so B is added.
- B –ε→ C, so C is added.
- C has no ε-moves.
- **ε-closure(A) = {A, B, C}**

ε-closure(B):

- B is in its own closure.
- B –ε→ C, so C is added.
- **ε-closure(B) = {B, C}**

ε-closure(C):

- C has no ε-transitions.
- **ε-closure(C) = {C}**

6. Step 2: The conversion formula

The transition function of the new NFA (without ε) is:

δ'(q, x) = ε-closure(δ(ε-closure(q), x)) for each state q and each x ∈ Σ.

Meaning, in three stages:

1. Take any number of ε-moves from q.
2. Read the symbol x.
3. Take any number of ε-moves again.

7. Step 3: Computing every new transition

For state A (ε-closure(A) = {A, B, C})

δ'(A, 0):

- δ(A, 0) = {B, C}
- δ(B, 0) = ∅
- δ(C, 0) = {C}
- Union = {B, C}
- ε-closure({B, C}) = {B, C} ∪ {C} = {B, C}

δ'(A, 1):

- δ(A, 1) = {A}
- δ(B, 1) = {B}
- δ(C, 1) = {C}
- Union = {A, B, C}
- ε-closure({A, B, C}) = {A, B, C}

For state B (ε-closure(B) = {B, C})

δ'(B, 0):

- δ(B, 0) = ∅
- δ(C, 0) = {C}
- Union = {C}
- ε-closure({C}) = {C}

δ'(B, 1):

- δ(B, 1) = {B}
- δ(C, 1) = {C}
- Union = {B, C}
- ε-closure({B, C}) = {B, C}

For state C (ε-closure(C) = {C})

δ'(C, 0):

- δ(C, 0) = {C}
- ε-closure({C}) = {C}

δ'(C, 1):

- δ(C, 1) = {C}
- ε-closure({C}) = {C}

8. Step 4: Finding the final states

A state q becomes a final state of the new NFA if ε-closure(q) contains an original final state. The

original final state is C.

- ε-closure(A) = {A, B, C} contains C, so A is final.
- ε-closure(B) = {B, C} contains C, so B is final.
- ε-closure(C) = {C} contains C, so C is final.

F' = {A, B, C}. The start state remains A.

9. Step 5: Transition table of the equivalent NFA (without ε)

State	0	1
→ * A	{B, C}	{A, B, C}
* B	{C}	{B, C}
* C	{C}	{C}

- → marks the initial state (A).
- * marks the final states (A, B and C).

10. Formal definition of the resulting NFA

N' = (Q, Σ, δ', A, F') where

- Q = {A, B, C}
- Σ = {0, 1}
- Start state = A
- F' = {A, B, C}
- δ' is as in the table above

11. Transition diagram of the new NFA (description)

- A has a transition on 0 to B and C, and a transition on 1 to A, B and C (including a self-loop on 1).
- B has a transition on 0 to C, and a transition on 1 to B (self-loop) and C.
- C has self-loops on both 0 and 1.
- A is the start state, drawn with an incoming arrow.
- A, B and C are all drawn with double circles.
- **No ε-edges** remain in the diagram.

12. Verification with sample strings

String	Original ε-NFA	New NFA	Result
ε	A –ε→ B –ε→ C (final)	A is final	Accepted
0	A –0→ C	δ'(A,0) = {B,C} contains finals	Accepted
1	A –ε→ B –ε→ C then 1 → C	δ'(A,1) = {A,B,C}	Accepted
01	A –0→ C –1→ C	{B,C} → on 1: {B,C} ∪ {C} = {B,C}	Accepted
10	A –1→ A, then A –0→ C	{A,B,C} → on 0: {B,C} ∪ {C} = {B,C}	Accepted
110	A –1→ A –1→ A –0→ C	{A,B,C} → {A,B,C} → {B,C}	Accepted

In every case the new NFA ends in a set containing final states, matching the original.

13. Observations

1. **Same number of states:** The new NFA has the same 3 states as the original. Only the ε-edges are removed, and new direct edges are added.
2. **Empty string:** Because ε-closure(A) contains the final state C, the start state A becomes final. So the empty string ε is accepted by both automata.
3. **Language accepted:** From A, the machine can reach C using only ε-moves (A → B → C),

and C loops on both 0 and 1. So every string over {0,1} is accepted. The language is **L = (0+1)*** (that is, Σ*). This holds under any reading of the 0-arrows from A, because the ε-path A → B → C alone is enough.

4. **Simplification:** Since the language is all of Σ*, a one-state NFA that is both initial and final, with self-loops on 0 and 1, would be equivalent. The conversion above is the systematic method, and it does not simplify automatically.

14. General algorithm summary

1. Compute ε-closure for every state.
2. For every state q and symbol x, compute δ'(q,x) = ε-closure(δ(ε-closure(q), x)).
3. Mark q as final if ε-closure(q) contains any original final state.
4. Keep the same start state.
5. Remove all ε-transitions.

15. Conclusion

The ε-closures are ε-closure(A) = {A,B,C}, ε-closure(B) = {B,C} and ε-closure(C) = {C}. The equivalent NFA without ε-transitions has the same states A, B and C, with initial state A and final states A, B and C. Its transition table is the one given in Section 9, and it accepts the same language as the original ε-NFA.

